

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline

# 1. Generate synthetic dataset
np.random.seed(42)
X = np.linspace(0, 6, 100).reshape(-1, 1)

# 2 & 3. Target variable:  $y = 2x^2 - 3x + \text{noise}$  (Gaussian noise: mean=0, std=0.7)
noise = np.random.normal(0, 0.7, 100).reshape(-1, 1)
y = 2 * (X**2) - 3 * X + noise

# Plot X vs y to understand the pattern
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Data Points', s=15)
plt.title('Synthetic Non-linear Dataset')
plt.xlabel('X')
plt.ylabel('y')

# Function to train and evaluate models
def evaluate_model(model, name):
    model.fit(X, y)
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)
    print(f"{name} -> MSE: {mse:.4f}, R2 Score: {r2:.4f}")
    return y_pred

# --- Linear Regression (Underfitting) ---
lr_model = LinearRegression()
y_pred_lr = evaluate_model(lr_model, "Linear Regression")
plt.plot(X, y_pred_lr, color='red', label='Linear (Underfit)')

# --- Polynomial Regression (Degree = 3) ---
poly3 = make_pipeline(PolynomialFeatures(3), LinearRegression())
y_pred_poly3 = evaluate_model(poly3, "Polynomial (Degree 3)")
plt.plot(X, y_pred_poly3, color='green', label='Poly Degree 3')

# --- Polynomial Regression (Degree = 40 - Overfitting) ---
poly40 = make_pipeline(PolynomialFeatures(40), LinearRegression())
y_pred_poly40 = evaluate_model(poly40, "Polynomial (Degree 40)")
plt.plot(X, y_pred_poly40, color='orange', label='Poly Degree 40')

```

```
(Overfit)')  
  
# --- Regularization Models (Applying on Degree 40 to see the effect)  
---  
ridge = make_pipeline(PolynomialFeatures(40), Ridge(alpha=1.0))  
evaluate_model(ridge, "Ridge Regression")  
  
lasso = make_pipeline(PolynomialFeatures(40), Lasso(alpha=0.1))  
evaluate_model(lasso, "Lasso Regression")  
  
elastic = make_pipeline(PolynomialFeatures(40), ElasticNet(alpha=0.1,  
ll_ratio=0.5))  
evaluate_model(elastic, "Elastic Net")  
  
plt.legend()  
plt.ylim(y.min()-5, y.max()+5) # To keep the view clear from Degree 40  
fluctuations  
plt.show()
```